

Informe 2: Verificación de sistemas digitales

Carlos Rubio, Santiago Cadena

cyrubiom@unal.edu.co, scadena@unal.edu.co Departamento de Ingeniería Eléctrica y Electrónica
Universidad Nacional de Colombia. Bogotá D.C., Colombia.

I. INTRODUCCIÓN

Se realizará el diseño y la implementación de un bus de datos como el sistema de comunicación más importante entre distintos elementos de un mismo sistema. El sistema tendrá maestros, esclavos, políticas de arbitraje y con esto, se analizará el desempeño al final variando la configuración.

A. Objetivos

- Con base a métricas, medir el comportamiento de un bus de datos en diferentes circunstancias de tráfico.
- Variar los distintos tipos de declaraciones en maestros y esclavos en un sistema de SystemC.
- Usar varias métricas para evaluar el ejecutable obtenido.

II. MARCO TEÓRICO

Los esclavos son dispositivos que responden a las solicitudes de los maestros. Pueden ser módulos de memoria o periféricos, y se clasifican en dos tipos: los esclavos de memoria rápida (como cachés y memorias RAM de alta velocidad), optimizados para un acceso inmediato, y los esclavos de memoria lenta (como memorias flash o discos), que requieren más tiempo para completar una transacción. La elección del tipo de esclavo tiene un impacto directo en el rendimiento del sistema, ya que una memoria lenta puede convertirse en un cuello de botella, especialmente en aplicaciones que requieren un acceso frecuente a los datos.

Un sistema de comunicación en la arquitectura de hardware digital depende de un bus de datos eficiente para permitir el intercambio de información entre los distintos componentes del sistema. En estos sistemas, los dispositivos maestros gestionan las solicitudes de acceso y la comunicación a través del bus, mientras que los dispositivos esclavos responden a dichas solicitudes. Para medir el rendimiento de estos sistemas, se utilizan diversas métricas: tales como el **throughput (rendimiento)**, el **tiempo de respuesta promedio** y la **utilización del bus**.

El bus de datos es la infraestructura que facilita la **transferencia de información** entre los diferentes módulos de un sistema digital. Existen diversos tipos de buses, que varían en función de su arquitectura y funcionalidad, como los buses paralelos y los buses seriales. En arquitecturas como **AMBA**

(Advanced Microcontroller Bus Architecture) de ARM, los buses se dividen en de alto rendimiento (como **AXI** y **AHB**) y de bajo consumo para periféricos (como APB). Un bus eficiente debe garantizar un acceso ágil a la memoria y reducir la latencia en la comunicación entre los dispositivos.

Los dispositivos maestros son los encargados de iniciar las transacciones en el bus. Estos pueden utilizar diferentes tipos de acceso. En el caso del **acceso blocking**, el maestro debe esperar a que la transacción termine antes de continuar con otra operación. En el acceso **non-blocking**, el maestro puede seguir realizando otras tareas mientras espera a que se complete la transacción. También existe el acceso **directo (direct access)**, donde el maestro tiene un canal exclusivo con el esclavo, sin necesidad de arbitraje. En sistemas avanzados, el bus puede gestionar varios maestros, lo que requiere un **mecanismo de arbitraje** para determinar cuál tiene prioridad en un momento dado.

Cuando múltiples maestros compiten por el acceso al bus, es necesario aplicar políticas de arbitraje para determinar el orden en que se les concede acceso. Existen diversas estrategias de arbitraje, cada una con ventajas y desventajas según la aplicación. Una de las más comunes es el arbitraje basado en prioridad fija, donde a cada maestro se le asigna un nivel de prioridad y el maestro de mayor prioridad obtiene acceso primero, aunque esto puede provocar inanición en los maestros de menor prioridad. Otra opción es el arbitraje **round-robin**, que asigna el acceso al bus de manera equitativa entre todos los maestros, evitando la inanición, pero sin optimizar el rendimiento en tareas críticas. En sistemas con alta concurrencia, se puede emplear el arbitraje dinámico basado en demanda, que ajusta la prioridad de los maestros en función de su frecuencia de uso o la latencia experimentada. También existe el arbitraje basado en lotería, en el que se asignan "tickets" a los maestros y el acceso al bus se determina aleatoriamente, lo que permite una distribución probabilística del recurso. La elección de la política de arbitraje depende de los requisitos específicos del sistema, priorizando la equidad, la latencia o el throughput, según la naturaleza de la aplicación.

Las métricas de desempeño son fundamentales para evaluar la eficiencia de un sistema basado en un bus. El testbench es una herramienta de simulación que permite analizar la funcionalidad y el rendimiento del bus en diversas condiciones. El **throughput**, o **tasa de transferencia**, mide la cantidad de datos transmitidos por unidad de tiempo y es esencial para determinar la eficiencia del bus. El tiempo de respuesta promedio evalúa el retraso entre la solicitud de una transacción

y su finalización, lo cual es crucial en **sistemas embebidos** y de **tiempo real**. Finalmente, la utilización del bus mide qué porcentaje del tiempo está ocupado el bus, lo que permite identificar posibles cuellos de botella o ineficiencias en el uso de los recursos de comunicación.

III. DESARROLLO

Para esta práctica se realizarán diferentes versiones de un mismo bus de datos variando la cantidad de elementos utilizados. Primero se utilizará un máster directo, después uno blocking y uno non-blocking como los elementos principales generadores de tráfico.

Una vez configurados, se estudian las métricas de desempeño como el nivel de utilización, throughput, duración promedio entre otras cosas que al final se pueden medir.

A. Primera Versión

Esta práctica está basada en una topología que se basa en módulos, interfaces, y agrupación de componentes junto con la definición de cada uno de ellos con el "maestro directo".

```
#include "systemc.h"
#include "simple_bus_test.h"

sc_time total_latency(0, SC_NS);
sc_time total_time(0, SC_NS);
unsigned int num_transactions = 0;

int sc_main(int, char **)
{
    simple_bus_test top("top");
    sc_clock clk("clk", 10, SC_NS); // Definir un reloj de 10 ns
    sc_signal<int> data_signal("data"); // Señal de datos

    sc_trace_file *tf;
    tf = sc_create_vcd_trace_file("simple_bus_trace"); // Nombre del archivo .vcd
    tf->set_time_unit(1, SC_PS); // Opcional: Define la escala del tiempo

    // Registra señales en la traza (ejemplo con una señal de reloj y de datos)
    sc_trace(tf, clk, "clock");
    sc_trace(tf, data_signal, "data");

    sc_start(10000, SC_NS);
    sc_time simulation_time = sc_time_stamp();
    double throughput = (num_transactions * 4) / simulation_time.to_seconds();
    double avg_latency = total_latency.to_seconds() / num_transactions;
    double response_time = total_latency.to_seconds() / num_transactions;
    double bus_utilization = total_latency.to_seconds() / 10e-6;

    cout << "=== Resultados de Desempeño ===" << endl;
    cout << "Tiempo total de simulación: " << simulation_time.to_string() << endl;
    cout << "Número total de transacciones: " << num_transactions << endl;
    cout << "Latencia promedio: " << avg_latency << " s" << endl;
    cout << "Throughput: " << throughput << " Bytes/s" << endl;
    cout << "Tiempo de respuesta promedio: " << response_time << " s\n";
    cout << "Nivel de utilización del bus: " << bus_utilization * 100 << " %\n";
    sc_close_vcd_trace_file(tf); // Cierra el archivo de trazas

    return 0;
}
```

Figure 1: Simple Bus Main Versión 1

B. Segunda versión

A diferencia de usar un maestro directo, en esta ocasión se usan métricas de comparación para modificar el código en lectura, escritura en: blocking, non blocking, y directo.

C. Tercera versión

En esta ocasión, se suman todas las latencias para de esta manera obtener una latencia acumulada pero cambiando

las operaciones de force read, force write por "READ" y "WRITE" comunes. De esta manera, se capturan las 4 sumas tanto del bloqueante como del no bloqueante, y se recopilan las sumas de las latencias en el system bus.

La latencia total y el número de transacciones capturadas se llevan a simple bus "main.cpp" donde se corre todo el código y se llaman todas las funciones para la simulación.

```
using namespace sc_core;
#include "systemc.h"
#include "simple_bus_test.h"

int sc_main(int, char **)
{
    simple_bus_test top("top");

    sc_start(10000, SC_NS);

    return 0;
}
```

Figure 2: Simple Bus Main.

```
#include "simple_bus_master_blocking.h"

void simple_bus_master_blocking::main_action()
{
    const unsigned int mylength = 0x10; // guarda la capacidad
    int mydata[mylength];
    unsigned int i;
    simple_bus_status status;

    while (true)
    {
        wait(); // ... para el proximo rising clock edge
        status = bus_port->burst_read(m_unique_priority, mydata,
                                     m_address, mylength, m_lock);
        if (status == SIMPLE_BUS_ERROR)
            sb_fprintf(stdout, "%s %s : lectura bloqueada en la dirección %x\n",
                      sc_time_stamp().to_string().c_str(), name(), m_address);

        for (i = 0; i < mylength; ++i)
        {
            mydata[i] += i;
            wait();

            status = bus_port->burst_write(m_unique_priority, mydata,
                                         m_address, mylength, m_lock);
            if (status == SIMPLE_BUS_ERROR)
                sb_fprintf(stdout, "%s %s : bloqueo de escritura failed at address %x\n",
                          sc_time_stamp().to_string().c_str(), name(), m_address);
        }

        wait(m_timeout, SC_NS);
    }
}
```

Figure 3: Master Blocking

```

#include "simple_bus_master_non_blocking.h"

void simple_bus_master_non_blocking::main_action()
{
    int mydata;
    int cnt = 0;
    unsigned int addr = m_start_address;

    wait(); // ... para el proximo rising clock edge
    while (true)
    {
        bus_port->read(m_unique_priority, &mydata, addr, m_lock);
        while ((bus_port->get_status(m_unique_priority) != SIMPLE_BUS_OK) &&
            (bus_port->get_status(m_unique_priority) != SIMPLE_BUS_ERROR))
            wait();
        if (bus_port->get_status(m_unique_priority) == SIMPLE_BUS_ERROR)
            sb_fprintf(stdout, "%s %s : ERROR cannot read from %x\n",
                sc_time_stamp().to_string().c_str(), name(), addr);

        mydata += cnt;
        cnt++;

        bus_port->write(m_unique_priority, &mydata, addr, m_lock);
        while ((bus_port->get_status(m_unique_priority) != SIMPLE_BUS_OK) &&
            (bus_port->get_status(m_unique_priority) != SIMPLE_BUS_ERROR))
            wait();
        if (bus_port->get_status(m_unique_priority) == SIMPLE_BUS_ERROR)
            sb_fprintf(stdout, "%s %s : ERROR cannot write to %x\n",
                sc_time_stamp().to_string().c_str(), name(), addr);

        wait(m_timeout, SC_NS);
        wait(); // ... para el proximo rising clock edge

        addr++; // next word (byte addressing)
        if (addr > (m_start_address+0x80)) {}
        addr = m_start_address; cnt = 0;
    }
}

```

Figure 4: Master Non-Blocking

D. Cuarta versión

Se realiza un análisis más detallado imprimiendo todas las transacciones con lo siguiente: registro, hora, nombre, ID de donde se hizo la operación.

E. Quinta versión

En esta ocasión se agregaron datos extra para el maestro blocking y non-blocking: banderas para identificar tanto operaciones realizadas como el maestro que ordena.

IV. CONCLUSIONES

- Se optimizaron los diferentes recursos encontrados en la librería de SystemC para optimizar los resultados.
- Se optimizó el rendimiento de SystemC debido además al diseño de múltiples sistemas propuestos.
- Debido a que la documentación de SystemC no fue tan clara desde el inicio, fue bastante arduo poder encontrar las herramientas necesarias para optimizar los recursos, especialmente la latencia promedio, throughput, y el nivel de utilización del bus.
- Non-blocking: Ideal para sistemas donde el maestro necesita solapar operaciones o ejecutar tareas en paralelo.
- Blocking: Más eficiente para transferencias grandes de datos (bursts) con menor sobrecarga de código.

REFERENCES

- [1] Infineon Technologies AG, *BFP460 Wideband Low Noise Transistor Datasheet*, Rev. 2.2, Nov. 2018. [Online]. Available: <https://www.infineon.com/part/BFP460#!?fileId=8ac78c8c7e7124d1017f0227976a6c9d>
- [2] Infineon Technologies AG, *Design Guide for Low Noise Transistor in FM Radio Front-End*, Application Note, Jan. 2004. [Online]. Available: <https://www.infineon.com/assets/row/public/documents/24/42/infineon-design-guide-for-low-noise-tr-in-fm-radio-fe-applicationnotes-en.pdf>
- [3] R. E. Collin, *Fundamentals of Microwave and RF Design*. Raleigh, NC: SciTech Publishing, 2001.